Implement the POST Endpoint for Creating a New Product

Discover how to implement a POST endpoint in a Python REST API with Flask to create new product entries. Learn to handle JSON request data, generate resource URIs dynamically, and respond with appropriate HTTP status and headers. This lesson guides you through step-by-step coding and testing for practical API development.

In a REST API, insert (aka create) is used to add a new resource to the existing collection. In this lesson, we'll see how to implement an API endpoint using POST, which adds/creates a new product to the existing collection of products.

The `request` object of Flask can be used to retrieve the body of a POST request. The `route` decorator can be used to map a method to the POST request. Let us see how to implement an API to add a new product by using the `request` object and the `route` decorator.

Steps

1. Open `server/api/products_api.py`.
2. Add a new method. Name the method `add_product`.

```
def add_product():
```

3. Add the following code.

Ask

```
data = request.get_json()
data['id'] = str(len(products) + 1)
```

```
products.append(data)
location = url_for('.get_product', product_id = data['id'], _external=True)
response = jsonify(dict(product=data, location=location), 201)
response.headers['Location'] = location
return response
```

4. Decorate the method with the `route` decorator of `products_blueprint`. Pass the URI that will represent the endpoint and the supported HTTP method, which is POST.

```
@products_blueprint.route('/products', methods=['POST'])
```

5. Run the application.
6. Hit the following URL using your preferred tool. When testing on the Educative platform, please replace `http://localhost` with the URL provided in the SPA widget.

```
$ curl -X POST --data '{"name":"rest-container","description":"container for
```

7. The obtained output should be as follows.

```
< HTTP/1.1 201 CREATED
< Content-Type: application/json
< Content-Length: 92
< Location: http://localhost:5000/api/products/3
{"location":"http://localhost:5000/api/products/3","product":{"id":"3","name"
```

How it works

Ask

Request/response for POST API to add a new product

The `request` object has a `get_json` method that returns the JSON passed as the body of the POST request. We've assigned the JSON object to the `data` variable, set its `id`, and added it to the existing list of products.

The response header, as well as the body, needs to have the URI using which the new product can be accessed. The URI can be created by using the `url_for` method of Flask. As arguments, it takes the method whose URL has to be created and the ID that needs to be passed as the path parameter. In our case, the method is `.get_product`, and the path parameter is the `id` of the new product. The URL thus created is set as the `location` header of the response returned by the `jsonify` method.

Ask

The `jsonify` method, by default, sets its status as `200 OK`. However, we need `201 CREATED` as the status. Hence, we pass `201` as one of the arguments of `jsonify`,

the other being the body of the response.

The `response` created as a result is returned, and the client receives the URI of the newly created product in the location header as well as the body. So, when we hit `/api/products` with the products data using POST, a new item is added to the products list, and the response is sent to the client.

Try it yourself

- The widget below shows the code created during the last lesson.
- Follow the steps above to complete the code for this lesson.
- Click “Run” in the widget below to start the service. The app URL for accessing the service is mentioned at the bottom of the widget.
- Wait for the service to start before accessing the URL.
- Append `/api/products` to the app URL and provide the JSON body to send the POST request. Set the `Content-Type` header as `application/json`.

> **Note:** Use REST API clients, such as Postman and Insomnia, or command line tools like curl to send the request.

- Send a GET request by appending `/api/products/3` to the app URL to verify that the update was successful.

> **Note:** Access the URL using any web browser, REST API clients, such as Postman and Insomnia, or command line tools like curl.

Ask

Run the code above, and access the URL once the service starts, to view the output

Solution

If stuck, refer to the final code below for help.

Please note that the file below is read-only. The changes from the previous lesson are highlighted.

Ask

products_api.py Python 3.8

```python
from flask import (request, jsonify, Blueprint, make_response, url_for)
from .data_store import products

products_blueprint = Blueprint('products_api', __name__, url_prefix='/api


@products_blueprint.route('/products', methods=['GET'])
def list_products():
    return jsonify(products)
```

Ask